Sprint 4 · Reto «Herramientas internas»

Ficha completa del reto, para consultar. Para trabajar, las tarjetas de misión (una por misión). Para entregar, entrega y rúbrica. Para contrastar lo hecho, el solucionario de las misiones.

- Volver al README del sprint · README del sprint en PDF · ecofriendly
- **Esta ficha en PDF:** a color · ecofriendly, para imprimir
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO **Ficha del reto**

1 Qué os pide el cliente

Qué os pide el cliente. El equipo de reservas de TurisData calcula cada semana sus cifras **a mano en una hoja de cálculo**: es lento y cada persona lo hace distinto. Os pide una **herramienta interna**, un programa en Python que lea el fichero de reservas, calcule **tres KPI** (cifras que resumen cómo va el negocio: la ocupación, el ingreso medio por reserva y la tasa de cancelación por canal), **avise con claridad si el fichero está mal** y guarde el resultado en un fichero JSON que otros sistemas puedan leer.

Cómo se trabaja. No hace falta experiencia previa: os damos una **plantilla casi hecha** (funciones vacías y pruebas que os dicen si lo que escribís es correcto) y vuestro trabajo es rellenarla misión a misión. La ocupación será solo un *proxy* (una medida aproximada), porque el fichero no trae la capacidad real del hotel. El reto se hace en **seis misiones** repartidas en dos días, cada una con sus pasos de **aprender** (unas lecciones del curso «Python desde cero» y unos puzles del Taller de bloques), seguidos de los de **construir** y **comprobar**: el **martes** la lógica con los datos en memoria y el **miércoles** los ficheros, los errores y la entrega. En clase tenéis unos **185 minutos de misiones el martes** (más 40 en la tutoría para acabar la misión 3) y **150 el miércoles** (misiones 4 y 5, README y entrega); lo que no quepa va a la tutoría o a casa. Cada persona hace además dos *katas* individuales de 10 minutos. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Python 3.11 (solo biblioteca estándar: `csv`, `json`, `argparse`, `datetime`)	Escribir y ejecutar `kpis.py`	Ya instalado en el aula; si no, el de Colab
Editor y terminal: VS Code (recomendado) o Colab	Editar el código y ejecutar `python test_kpis.py`	En Colab, subid la carpeta y poned `!` delante: `!python test_kpis.py`
La plantilla y el CSV real: `reto_herramientas.zip`	Una sola descarga con la carpeta `reto_herramientas/` y el CSV `reservas_turisdata.csv` (6000 reservas **sintéticas**) dentro: `kpis.py` con funciones vacías y pasos, `test_kpis.py` (por fases), `README_PLANTILLA.md`, los archivos de las misiones 0 a 2 (`demo_cliente.py`, `reservas_demo.py`, `mision_1_datos.py`, `mision_2_decidir.py`) y `extra/plantilla_kpis_pandas.py` (solo para el nivel extra)	Descomprimidlo y trabajad ahí: el CSV ya está **junto a `kpis.py`**
Las tarjetas de misión (índice)	Una por misión, para tenerla abierta mientras trabajáis: qué lograréis, la idea en un minuto, el paso a paso explicado, cuándo está terminada y qué hacer si os atascáis	En el repositorio del curso, en `1_Trabajar/tarjetas/`
El solucionario de las misiones (abrir)	Contrastar vuestro trabajo cuando hayáis terminado una misión o agotado las pistas: soluciones de las misiones 0 a 2 y cifras para comprobar a mano las 3 a 5	En esta carpeta; se publica por días (martes: misiones 0 a 2; miércoles: 3 a 5)
El curso «Python desde cero» (abrir) y **el Taller de bloques** (abrir)	Los pasos de «aprender» de cada misión (lecciones mínimas y puzles) y las dos katas	En el navegador, sin instalar nada; podéis exportar vuestro progreso
Cuadernos del sprint: `S04_03` y `S04_04`	Las pistas para las funciones (diccionarios, CSV, JSON, excepciones)	Colab; su código explicado

Recurso	Para qué lo usaréis	Dónde está
Git y GitHub + cuaderno `S04_05`	Guardar versiones (*commits*) y entregar	Terminal o VS Code. Si falla, subid los ficheros desde la web (*Add file → Upload files*): cada subida es un commit
Entrega y rúbrica	Qué se sube, qué se evalúa, el README y la subida a `sprint-04/`; cada persona rellena su autoevaluación y coevaluación	`entrega_y_rubrica.md` y la guía de entrega
Un asistente de IA (opcional)	Entender errores y pedir pistas	Con las reglas de la «Ayuda 1» del apartado 4

3 · Qué entregar

En total, al final del sprint se suben a `sprint-04/`: 4 ficheros del equipo (`kpis.py`, `test_kpis.py`, `README.md`, `salida.json`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **4 + tantos ficheros como personas haya en el equipo** (un equipo de 3 sube 7). «El reto» no es un fichero aparte: son esos cuatro ficheros. Además, el CSV va a `data/` si aún no está, y los extras solo si los hacéis. La *review* y la *retro* se hacen de palabra: no se suben.

Qué ficheros se suben y de dónde sale cada uno, qué requisitos se evalúan (mínimo, estándar, extra), la rúbrica y cómo se revisa: **Entrega y rúbrica**.

4 · Guía paso a paso: las tarjetas de misión

El paso a paso de las **seis misiones** (qué hacéis, cómo se hace, cuándo está terminada y qué hacer si os atascáis) está en las **tarjetas de misión**: una por misión, para tener abierta solo la que toca. Aquí queda únicamente lo que vale para todo el reto:

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da código que parece funcionar y no se entiende**. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea, tenéis que saber explicarla.
- **Las pruebas mandan:** si el código de la IA no las pasa, no vale.
- **Declaradlo** en la sección «Uso de IA y límites» del README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender un error	«Me sale este error: *[mensaje completo]*. Explícame qué significa, sin darme todavía la solución»	Que lo entendéis y lo arregláis vosotros
Entender una función	«Explícame línea por línea qué hace esta función: *[código]*»	Ejecutadla con un ejemplo pequeño
Pedir una pista	«Quiero calcular el % de cancelaciones por canal con la biblioteca estándar. ¿Qué estructura me conviene? No me des el código»	Que respeta «solo biblioteca estándar»
Revisar el estilo	«¿Sigue PEP 8 este código? ¿Qué cambiarías?»	Que las pruebas siguen pasando